

Docker Oracle 설치 교본

Rocky Linux + Docker Compose · Oracle AI Database Free · 초보자용

목표: Oracle 공식 Free 컨테이너 이미지를 사용하여 Docker Compose로 Oracle을 설치하고, 데이터 영구저장, 1521 외부접속, devuser 사용자/스키마 생성, DBeaver 연동까지 구성함.

1. 이 교본에서 사용하는 Oracle

Oracle 공식 컨테이너 이미지 `container-registry.oracle.com/database/free:latest` 를 사용함. 기본 컨테이너 데이터 경로는 `/opt/oracle/oradata` 이며, 기본 서비스는 `FREEPDB1`을 사용함.

Oracle AI Database Free는 학습·개발에 편리하지만 Free 에디션의 CPU·메모리·사용자 데이터 용량 제한 및 지원 정책이 있으므로 대규모 상용 운영은 별도 라이선스/지원 정책을 검토해야 함.

2. 최종 디렉터리 구조

```
/home/devuser/docker/oracle/
├── compose.yml
├── .env
└── data/                ← 실제 Oracle DB 데이터 영구 저장
```

MariaDB/PostgreSQL과 마찬가지로 실제 DB 데이터는 `~/docker/oracle/data` 에 두어 컨테이너와 분리함.

3. Docker 설치 확인

```
docker --version
docker compose version
sudo systemctl status docker
```

```
sudo systemctl enable --now docker
```

4. 기존 Oracle/1521 포트 확인

```
sudo ss -lntp | grep 1521
docker ps -a
```

1521을 이미 다른 Oracle이나 컨테이너가 사용 중이면 먼저 해당 서비스를 확인해야 함.

5. Oracle 작업 폴더 만들기

```
mkdir -p ~/docker/oracle/data
cd ~/docker/oracle
pwd
ls -la
```

6. .env 만들기

```
nano .env
```

bash

```
ORACLE_PASSWORD=본인이정한강력한비밀번호  
ORACLE_PORT=1521
```

env

```
chmod 600 .env
```

bash

ORACLE_PASSWORD는 SYSTEM/SYS 관리 계정 접속에 사용하는 비밀번호임. 충분히 강한 비밀번호를 사용함.

7. compose.yml 만들기

```
nano compose.yml
```

bash

```
services:  
  oracle:  
    image: container-registry.oracle.com/database/free:latest  
    container_name: wiz-oracle  
    restart: unless-stopped  
  
    environment:  
      ORACLE_PWD: ${ORACLE_PASSWORD}  
  
    ports:  
      - "${ORACLE_PORT}:1521"  
  
    volumes:  
      - ./data:/opt/oracle/oradata:Z  
  
    healthcheck:  
      test: ["CMD-SHELL", "healthcheck.sh"]  
      interval: 30s  
      timeout: 10s  
      retries: 10  
      start_period: 5m
```

yaml

핵심은 ./data:/opt/oracle/oradata:Z 임. Rocky Linux SELinux 환경을 고려해 :Z를 사용함. Oracle 초기화는 PostgreSQL/MariaDB보다 오래 걸릴 수 있음.

8. Compose 문법 검사

```
docker compose config
```

bash

YAML 오류가 없을 때만 다음 단계로 진행함. TAB 대신 공백을 사용함.

9. Oracle 이미지 받기 및 시작

```
docker compose pull  
docker compose up -d  
docker ps
```

bash

Oracle 공식 이미지는 크기가 크므로 최초 다운로드와 DB 초기화에 시간이 걸릴 수 있음.

10. Oracle 초기화 로그 확인

```
docker logs -f wiz-oracle
```

bash

로그에서 DATABASE IS READY TO USE! 와 같은 준비 완료 메시지가 나타날 때까지 기다림. 로그 보기 종료는 Ctrl+C이며 컨테이너는 계속 실행됨.

11. 1521 포트 확인

```
sudo ss -lntp | grep 1521
```

bash

정상이라면 0.0.0.0:1521 또는 [::]:1521에서 docker-proxy가 LISTEN 중임.

12. SYSTEM으로 Oracle 접속

```
docker exec -it wiz-oracle sqlplus system/"$ORACLE_PWD"@FREEPDB1
```

bash

위 방식은 셸 환경에 ORACLE_PWD가 없을 수 있으므로 초보자는 다음처럼 컨테이너 내부 셸에 들어가 접속하는 방식도 편리함.

```
docker exec -it wiz-oracle bash
sqlplus system@FREEPDB1
```

bash

비밀번호를 묻는 경우 .env에서 설정한 ORACLE_PASSWORD를 입력함.

```
SELECT sys_context('USERENV','CON_NAME') AS container_name FROM dual;
SHOW USER;
EXIT;
```

sql

13. devuser 사용자(스키마) 생성

Oracle에서 USER는 로그인 계정이면서 기본적으로 같은 이름의 스키마와 연결됨. SYSTEM으로 FREEPDB1에 접속한 상태에서 실행함.

```
CREATE USER devuser IDENTIFIED BY "강력한비밀번호";

GRANT CREATE SESSION TO devuser;
GRANT CREATE TABLE TO devuser;
GRANT CREATE VIEW TO devuser;
GRANT CREATE SEQUENCE TO devuser;
GRANT CREATE PROCEDURE TO devuser;

ALTER USER devuser QUOTA UNLIMITED ON USERS;
```

sql

학습용으로 필요한 일반 개발 권한을 부여한 예임. DBA 권한을 무조건 주는 것보다 필요한 권한만 부여하는 것이 안전함.

14. devuser에게 DBA 권한이 꼭 필요한 경우

```
GRANT DBA TO devuser;
```

sql

DBA는 매우 강한 관리자 권한이므로 개인 학습 서버에서는 사용할 수 있지만 실제 서비스 애플리케이션 계정에는 권장하지 않음.

15. devuser으로 접속

```
docker exec -it wiz-oracle sqlplus devuser@FREEPDB1
```

bash

비밀번호 입력 후 다음으로 확인함.

```
SHOW USER;  
SELECT sys_context('USERENV','CON_NAME') FROM dual;
```

sql

16. users 예제 테이블

```
CREATE TABLE users (  
    id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    username VARCHAR2(50) NOT NULL UNIQUE,  
    password VARCHAR2(255) NOT NULL,  
    name VARCHAR2(100) NOT NULL,  
    email VARCHAR2(150) UNIQUE,  
    phone VARCHAR2(20),  
    role VARCHAR2(20) DEFAULT 'USER',  
    enabled NUMBER(1) DEFAULT 1,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

sql

```
INSERT INTO users  
(username, password, name, email, phone, role)  
VALUES  
( 'hong', '1234', '홍길동', 'hong@test.com', '010-3333-4444', 'USER');  
  
COMMIT;
```

sql

실제 로그인 서비스에서는 password를 평문으로 저장하지 않고 BCrypt 등으로 해시하여 저장해야 함.

17. 테이블 조회

```
SELECT table_name FROM user_tables;  
DESC users;  
SELECT * FROM users;
```

sql

SQL*Plus에서 긴 결과가 불편하면 LINESIZE와 PAGESIZE를 조정할 수 있음.

```
SET LINESIZE 200  
SET PAGESIZE 100  
SELECT * FROM users;
```

sql

18. Rocky Linux 방화벽

내부망 192.168.0.0/24에서만 Oracle 1521을 허용하는 구성을 권장함.

```
sudo firewall-cmd --permanent --remove-port=1521/tcp
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="192.168.0.0/24" port port="1521" protocol="tcp" accept'
sudo firewall-cmd --reload
sudo firewall-cmd --list-all
```

기존에 1521/tcp 전체 허용이 없으면 remove-port 명령이 실패하거나 변경 없음으로 나올 수 있으며, 그 경우 rich rule 추가 여부를 확인하면 됨.

19. Windows에서 1521 테스트

현재 Rocky Linux 서버 내부 IP SERVER_IP 기준:

```
Test-NetConnection SERVER_IP -Port 1521
```

TcpTestSucceeded : True이면 Windows에서 Docker Oracle listener까지 연결됨.

20. DBeaver 연결

항목	값
DB 종류	Oracle
Host	SERVER_IP
Port	1521
접속 방식	Service Name
Service Name	FREEPDB1
Username	devuser
Password	devuser 생성 시 설정한 비밀번호

```
jdbc:oracle:thin:@//SERVER_IP:1521/FREEPDB1
```

SID가 아니라 Service Name=FREEPDB1을 사용하는 것이 핵심임.

21. 데이터 영구저장 원리

```
Rocky Linux
/home/devuser/docker/oracle/data
    |
    | Bind Mount
    ▼
Oracle Container
/opt/oracle/oradata
```

docker compose down으로 컨테이너를 제거해도 data 폴더는 남음. 같은 compose.yml로 다시 올리면 기존 데이터 파 bash 일을 사용함.

주의:

~/docker/oracle/data를 삭제하면 실제 Oracle 데이터 파일을 잃을 수 있으므로 함부로 삭제하지 않음.

text

22. 자주 쓰는 Docker 명령

목적	명령
설정 검사	docker compose config
이미지 받기	docker compose pull
시작	docker compose up -d
상태	docker ps
로그	docker logs wiz-oracle
실시간 로그	docker logs -f wiz-oracle
재시작	docker compose restart
중지	docker compose stop
컨테이너 제거	docker compose down
SYSTEM 접속	docker exec -it wiz-oracle sqlplus system@FREEPDB1
devuser 접속	docker exec -it wiz-oracle sqlplus devuser@FREEPDB1

23. 백업 기초

Oracle은 Data Pump(expdp/impdp), RMAN 등 여러 백업 방식이 있음. 초보자 단계에서는 DB 데이터 디렉터리를 단순 복사하는 것만으로 운영 중인 Oracle의 일관된 백업을 보장한다고 생각하면 안 됨. 실제 운영 백업은 RMAN 또는 Data Pump를 학습하여 구성하는 것을 권장함.

24. MariaDB/PostgreSQL과 다른 점

항목	MariaDB/PostgreSQL	Oracle Free
기본 포트	3306 / 5432	1521
DB 접속 단위	Database	PDB Service
기본 서비스	wizdb 등	FREEPDB1
사용자	계정 중심	사용자와 스키마가 밀접
데이터 경로	/var/lib/...	/opt/oracle/oradata
초기 시작	비교적 빠름	상대적으로 오래 걸림
관리 CLI	mariadb / psql	sqlplus

25. 초보자가 기억할 핵심 명령

```
cd ~/docker/oracle
docker compose config
docker compose up -d
docker logs -f wiz-oracle
sudo ss -lntp | grep 1521
docker exec -it wiz-oracle sqlplus system@FREEPDB1
docker exec -it wiz-oracle sqlplus devuser@FREEPDB1
docker compose down
```

bash

26. 최종 점검표

점검 항목	정상 기준
Compose	docker compose config 오류 없음
컨테이너	wiz-oracle 실행 중
초기화	DB 준비 완료 로그 확인
영구저장	~/docker/oracle/data 사용
Listener	1521 LISTEN
PDB	FREEPDB1
사용자	devuser 접속 가능
방화벽	192.168.0.0/24 → 1521 허용
Windows 테스트	TcpTestSucceeded : True
DBeaver	Service Name FREEPDB1 연결

27. 참고: Oracle Free의 성격

Oracle 공식 Free 컨테이너 이미지는 학습·개발·프로토타이핑에 매우 편리함. Free 에디션은 리소스 제한이 있으며 별도 지원/패치 정책도 Enterprise/상용 지원 제품과 다르므로 중요 상용 시스템에서는 라이선스와 지원 요건을 별도로 검토함.

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME...)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.